

Bridge Pattern in Java

Assignment 3 Shape and Renderer

Student: Saule Jumabay

Course: ShP-2216 Software Design Patterns

Astana IT University

Language: Java 17

1 Introduction

This project implements the Bridge structural design pattern using two shapes and two rendering implementations. A circle or square can use a vector renderer or a raster renderer. The program changes the renderer of each existing shape at runtime while keeping its geometry unchanged.

The design follows the Shape-Renderer example in Lecture 4, page 14, and extends it with a square, a runtime setter, shared input validation and automated behavior checks. Console output simulates drawing, as in the lecture; the project does not create a graphics window.

Why Bridge fits

There are two dimensions of variation: the shape being drawn and the drawing implementation. Combining these dimensions through inheritance would lead to classes such as VectorCircle, RasterCircle, VectorSquare and RasterSquare. Bridge lets the client combine independent shape and renderer objects instead.

Pattern role	Project component	Responsibility
Abstraction	Shape	Stores common geometry and the renderer reference.
Refined abstractions	Circle and Square	Convert their geometry into drawing operations.
Implementor	Renderer	Declares circle and line operations.
Concrete implementors	VectorRenderer and RasterRenderer	Interpret the operations for the selected drawing model.
Client	Main	Creates the objects and demonstrates switching.

Implementation choice

Renderer exposes renderCircle and renderLine rather than one method for each polygon. Square delegates four line operations. A triangle can therefore reuse the same renderer interface, making independent extension visible in this small example.

2 UML and collaboration

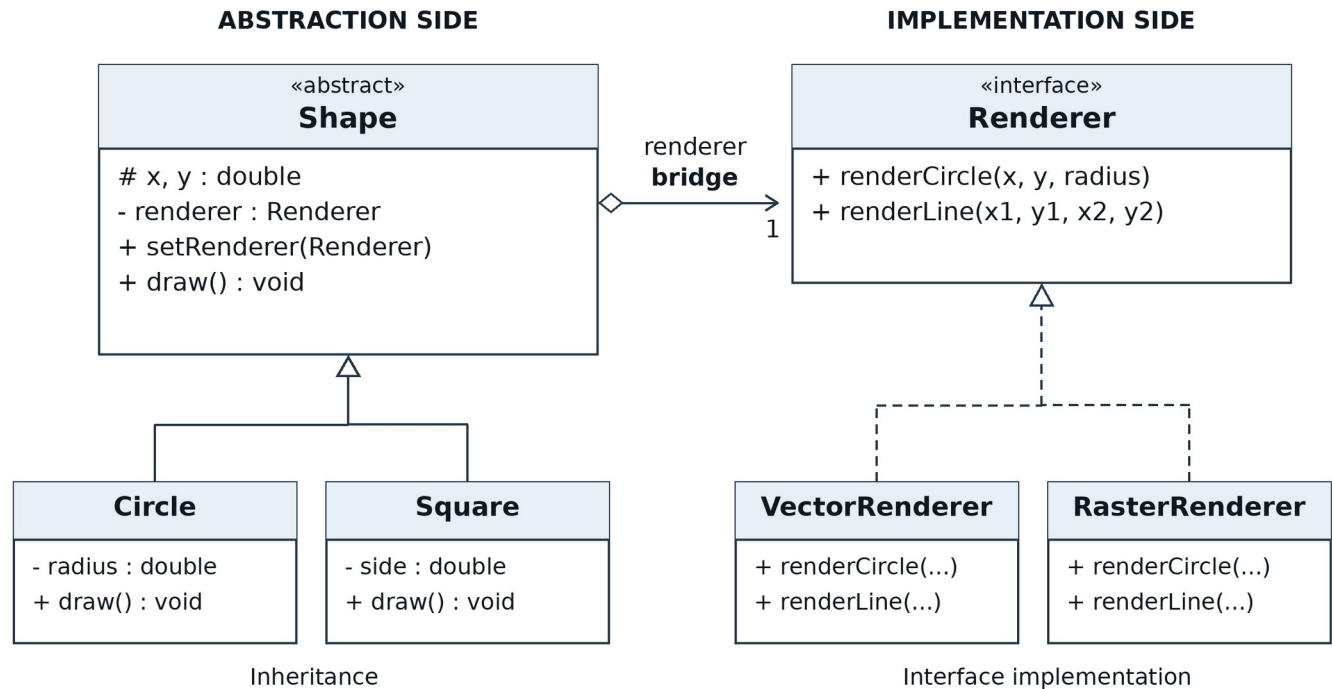


Figure 1. The two Bridge hierarchies. Constructors and validation helpers are omitted for readability. Main is the client that creates and combines the concrete objects.

The hollow diamond follows the lecture notation: Shape holds a shared Renderer reference. This is object composition in the design-pattern sense. It is not exclusive lifecycle ownership; Main can give the same renderer to several shapes. The interface has no reference back to Shape.

How a call travels through the bridge

Main creates a Circle with a VectorRenderer and calls draw(). Circle forwards its coordinates and radius to the Renderer interface. Dynamic dispatch selects VectorRenderer.renderCircle(). After setRenderer(raster), the next draw() on the same Circle delegates to RasterRenderer instead.

```
Shape circle = new Circle(5.4, 3.2, 2.6, vector);
circle.draw();           // delegates to VectorRenderer
circle.setRenderer(raster); // replaces the stored reference
circle.draw();           // same Circle, RasterRenderer
```

For Square, draw() computes the four edges from x, y and side. Both renderers receive the same endpoints. VectorRenderer prints decimal coordinates; RasterRenderer maps them to integer pixel coordinates using Math.round. A positive circle radius is represented by at least one pixel.

Inheritance exists within the shape hierarchy, while the renderer classes implement a common interface. The bridge between those two sides is the stored object reference, not inheritance.

3 Clean Code principles

3.1 Single responsibility

Circle owns circle geometry; it does not decide how pixels or vector commands are represented. Renderers own that interpretation, and Main owns the demonstration flow. Small classes keep changes close to the responsibility they affect.

```
// Circle.java: describe the shape and delegate the operation.
@Override
public void draw() {
    getRenderer().renderCircle(x, y, radius);
}
```

For example, a change in pixel rounding belongs in RasterRenderer and requires no change to Circle. The geometry calculation for four square edges belongs in Square, so it is not copied into both renderers.

3.2 Meaningful names

Names distinguish the high-level shape operation from low-level rendering operations. The terms radius, side, startX and endX express geometry directly. This makes method calls understandable without comments that repeat each statement.

```
// Renderer.java: operation names describe the required work.
void renderCircle(double x, double y, double radius);
void renderLine(double startX, double startY,
                double endX, double endY);
```

In Main, demonstrateSwitch names the purpose of the helper. VectorRenderer and RasterRenderer identify the implementation dimension, while Circle and Square identify the abstraction dimension.

3.3 Do not repeat yourself

The positive-size rule is shared in Shape rather than repeated in the two shape constructors. Renderer replacement is also implemented once in Shape and inherited by both shapes.

```
// Circle constructor:
this.radius = requirePositive(radius, "Radius");
// Square constructor:
this.side = requirePositive(side, "Side");
// One shared rule in Shape:
protected static double requirePositive(double value, String name) {
    requireFinite(value, name);
    if (value <= 0) {
        throw new IllegalArgumentException(name + " must be positive");
    }
    return value;
}
```

3 Clean Code principles continued

3.4 Dependency inversion

The abstraction depends on `Renderer`, an interface, rather than `VectorRenderer` or `RasterRenderer`. Each shape works through that contract. Main may know the concrete classes because it assembles the application, but geometry classes do not import or check them.

```
// Shape.java: depend on the rendering contract.
private Renderer renderer;

public final void setRenderer(Renderer renderer) {
    this.renderer = Objects.requireNonNull(
        renderer, "Renderer must not be null");
}
```

There are no instanceof checks or renderer-type branches in `Circle` or `Square`. The recording renderer in the tests can replace the production implementations without modifying the shapes. The setter rejects null before changing the current reference.

3.5 Open closed design

A new renderer can implement the existing two operations without changing `Shape`, `Circle` or `Square`. A new shape can extend `Shape` and compose the existing operations without changing either production renderer. The tests demonstrate both directions with `RecordingRenderer` and a test-only `Triangle`.

```
// Test-only Triangle.draw(): reuse the existing contract.
@Override
public void draw() {
    getRenderer().renderLine(x, y, x + 2, y);
    getRenderer().renderLine(x + 2, y, x + 1, y + 2);
    getRenderer().renderLine(x + 1, y + 2, x, y);
}
```

This independence has a boundary: the interface must already express the required drawing operations. A new feature such as an unsupported curve primitive could require extending `Renderer` and updating its implementations. Bridge reduces coupling; it does not remove every possible contract change.

Additional code quality decisions

Coordinates and dimensions are final, so switching a renderer does not mutate geometry. Constructors reject non-finite coordinates and non-positive or non-finite sizes. Production code contains seven small files and uses only the Java standard library. The test-only helpers remain outside `src`.

4 Demonstration and verification

The project was compiled with Java 17.0.20 using --release 17, -Xlint:all and -Werror. Compilation succeeded without warnings. Main demonstrates Circle and Square with both renderers; the assignments in setRenderer occur during the same program execution.

```
javac --release 17 -Xlint:all -Werror -d out src/bridge/*.java
java -cp out bridge.Main
```

The circle section of the actual output is:

```
=== Circle ===
Before switch:
Vector circle: center=(5.4, 3.2), radius=2.6
Switching renderer on the same Circle object...
After switch:
Raster circle: centerPixel=(5, 3), radiusPixels=3
```

The square section prints four vector edges and then four raster edges. The continuous square starts at (1.2, 2.4), with side 3.5; its opposite corner is (4.7, 5.9). Pixel endpoints are rounded independently. Full output is stored in docs/sample-output.txt.

Test group	Behavior checked	Result
Circle delegation	Exactly one circle call with unchanged geometry.	Pass
Square geometry	Four correct connected edges, including closure.	Pass
Runtime switching	New target receives calls; geometry stays unchanged; switching back works.	Pass
Null renderer	Constructor and setter reject null; old target remains usable.	Pass
Invalid geometry	Zero, negative and non-finite sizes; non-finite coordinates.	Pass
Vector output	Decimal coordinates and square closure.	Pass
Raster output	Rounded coordinates and minimum one-pixel radius.	Pass
Independent extension	New test shape and renderer work without editing production classes.	Pass
Client demonstration	All four shape-renderer combinations run in order.	Pass

Test result: PASS: 9 test groups, 41 checks. Run test.bat on Windows or sh test.sh on macOS/Linux. These tests verify the console simulation and delegation contract; they do not claim to test a real graphics engine.

5 Conclusion

The project separates shape geometry from the rendering implementation. Both Circle and Square use the same Renderer contract and support runtime replacement. The line-based square and the test-only triangle demonstrate how new shapes can reuse existing low-level operations.

Trade offs and limitations

The main benefit is flexibility: a new renderer does not require new versions of every shape. The cost is additional classes, an interface and a delegation step. For a program with only one permanent rendering method, this structure could be unnecessary.

Using a small primitive interface improves reuse but moves polygon geometry into the shape classes. Raster rounding can lose subpixel detail and can make a very small edge collapse to one pixel position. That is an intentional limitation of the console simulation. The renderers print descriptions; they do not produce vector files or raster images.

A larger application could replace the console implementations with real drawing backends while retaining supported shape operations. No performance measurements were taken, so the project does not make speed or memory claims.

Distinction from related patterns

Pattern	Main purpose
Bridge	Separate abstraction and implementation so both can vary through a shared contract.
Adapter	Make an existing incompatible interface usable through the interface a client expects.
Strategy	Make alternative algorithms or behaviors interchangeable for a context.

Bridge and Strategy may look similar because both use composition. Here the intent is to maintain a family of shapes independently of a family of rendering implementations, which is the Bridge use case described in the lecture.

6 Repository

GitHub repository: [INSERT YOUR REPOSITORY URL]

7 References

1. Ospanov, Almas. Lecture 4: The Structural Design Pattern – Bridge Pattern. Course lecture PDF, 18 pages. Component roles and UML: pp. 6–9; related patterns: pp. 12–13; Shape-Renderer code: p. 14; benefits and summary: pp. 17–18.
2. Astana IT University. Assignment 3 — Bridge Pattern. ShP-2216 Software Design Patterns. Technical requirements, Clean Code requirements, deliverables and grading criteria.